ĐẠI HỌC BÁCH KHOA HÀ NỘI

Trường Công nghệ Thông tin và Truyền thông



BÁO CÁO BÀI TẬP LỚN

Efficient RAG

Nén ngữ cảnh và truy hồi hiệu quả cho hỏi đáp với LLM

Đề tài 13: Efficient LLM/RAG – Token Compression, Context Compression và Efficient Retrieval

Sinh viên	Nguyễn Hoàng Sơn ⟨ 20261127M ⟩
	Võ Việt Hoàng ⟨ 20251064M ⟩
Giảng viên	Lê Thanh Hương
Học phần	IT4772 – Xử lý ngôn ngữ tự nhiên
Mã nguồn	https://github.com/hotamago/nlp-efficient-rag
Thời gian	Hà Nội, tháng 10 năm 2026

Tóm tắt

Hệ thống RAG đưa hàng chục đoạn văn vào LLM cho mỗi câu hỏi, khiến chi phí và độ trễ tăng theo số token, trong khi nhiều đoạn chỉ là nhiễu. Báo cáo này đánh giá có hệ thống hai hướng giảm chi phí đó: *nén ngữ cảnh* (cắt bớt, chọn tài liệu, chọn câu, xoá token, nén mềm thành vector) và *truy hồi hiệu quả* (BM25S, embedding tĩnh, Matryoshka, lượng tử int8/nhị phân, HNSW). Mọi phương pháp được đo trên cùng giao thức NQ-Open 20 tài liệu của *Lost in the Middle* (500 câu hỏi, đoạn chứa đáp án ở vị trí 1, 10, 20), cùng một reader Qwen2.5-7B-Instruct, cùng bộ đếm token, kèm kiểm định bootstrap ghép cặp.

Ở mức nén ≈4×, chọn lọc theo câu bằng Provence giữ 91,6% bằng chứng và đạt 64,0% accuracy so với 65,3% của ngữ cảnh đầy đủ (giảm 78% token, không có ý nghĩa thống kê), trong khi xoá theo token (LLMLingua-2) làm accuracy rơi về mức không có ngữ cảnh. Khác với kết quả công bố trên GPT-3.5-Turbo, reader 7B hiện đại gần như không còn hiện tượng *lost in the middle* ở 3K token (chênh 5,8 so với 21,6 điểm), nên nén không còn "làm phẳng" đường cong vị trí. Về truy hồi, một retriever dày đặc 33M tham số vượt BM25 21 điểm hit@5 và lượng tử int8 giảm chỉ mục 4× không mất chất lượng; ghép end-to-end, top-20 của retriever này cho accuracy ngang 20 tài liệu gốc (66,0%). Toàn bộ pipeline chạy được không cần GPU: bộ nén trên CPU (0,2–4,9 s/câu) và reader qua API. Nén mềm thành vector (PISCO, 18×) mất 9–17 điểm so với văn bản trên cùng reader; bộ nén encode-once nhóm tự huấn luyện trên Qwen2.5-1.5B không vượt cổng dừng đặt trước, và chẩn đoán cho thấy 8 vector/đoạn không giữ được đáp án. Trên reader 1.5B nhạy với vị trí, nén văn bản bằng Provence lại tăng accuracy 7,5 điểm. Mã nguồn, kết quả từng câu và demo Gradio được công khai.

Từ khoá: RAG, nén ngữ cảnh, prompt compression, truy hồi hiệu quả, lost in the middle, Provence, LLMLingua.

Mục lục

Tóm tắt **i**

1 Giới thiệu **1**
1.1 Bối cảnh . . . 1
1.2 Bài toán . . . 1
1.3 Câu hỏi nghiên cứu và đóng góp . . . 1

2 Nghiên cứu liên quan **2**
2.1 Nén cứng ở mức token . . . 2
2.2 Nén cứng ở mức câu/đoạn (context pruning) . . . 2
2.3 Nén mềm, bộ nhớ ngữ cảnh và KV-cache . . . 3
2.4 Truy hồi hiệu quả . . . 3
2.5 Bộ dữ liệu và giao thức đánh giá . . . 3

3 Phương pháp **3**
3.1 Kiến trúc tổng thể . . . 3
3.2 Các phương pháp nén ngữ cảnh . . . 4
3.3 Nén mềm "encode-once" (phương pháp đề xuất) . . . 5
3.4 Các phương pháp truy hồi hiệu quả . . . 7
3.5 Giao thức đánh giá . . . 7

4 Cài đặt hệ thống **8**
4.1 Tổ chức mã nguồn . . . 8
4.2 Thư viện, mô hình và dữ liệu sử dụng . . . 8
4.3 Thông số thực nghiệm . . . 8
4.4 Demo . . . 9

5 Kết quả và thảo luận **9**
5.1 Bộ nén giữ được bao nhiêu bằng chứng? . . . 9
5.2 Độ chính xác của RAG sau nén . . . 11
5.3 Nén và hiện tượng *lost in the middle* . . . 12
5.4 Đối chiếu với số liệu đã công bố . . . 13
5.5 Nén mềm . . . 14
5.6 Truy hồi hiệu quả . . . 16
5.7 RAG end-to-end . . . 17
5.8 Chạy không cần GPU . . . 17
5.9 Phân tích lỗi . . . 18

6 Kết luận **19**
6.1 Trả lời câu hỏi nghiên cứu . . . 19

6.2 Hạn chế 19
6.3 Hướng phát triển 20

Phân công công việc **20**

A Tái lập kết quả **23**

B Prompt **24**

C Siêu tham số mô hình nén mềm **24**

1 Giới thiệu

1.1 Bối cảnh

Retrieval-Augmented Generation (RAG) [23] là cách phổ biến nhất hiện nay để đưa tri thức bên ngoài vào mô hình ngôn ngữ lớn (LLM): với mỗi câu hỏi, một bộ truy hồi (retriever) tìm k đoạn văn liên quan trong kho tài liệu, rồi LLM sinh câu trả lời dựa trên các đoạn đó. Để không bỏ sót bằng chứng, hệ thống thường lấy k khá lớn (10–20 đoạn). Điều này gây ra ba vấn đề:

1. **Chi phí và độ trễ.** Prompt dài hàng nghìn token; chi phí API và thời gian xử lý prompt (prefill) tăng tuyến tính (thậm chí bậc hai với attention) theo độ dài ngữ cảnh.
2. **Nhiễu.** Phần lớn các đoạn truy hồi không chứa đáp án; chúng có thể đánh lạc hướng LLM.
3. **Thiên lệch vị trí.** Liu et al. [28] chỉ ra hiện tượng *Lost in the Middle*: độ chính xác của LLM cao khi đoạn chứa đáp án nằm ở đầu hoặc cuối prompt, nhưng giảm mạnh khi nó nằm ở giữa (đường cong hình chữ U).

Hướng *Efficient RAG* giải quyết các vấn đề trên ở hai đầu của pipeline: (i) **nén ngữ cảnh** (context/prompt compression) – loại bỏ token, câu hoặc đoạn không cần thiết trước khi đưa vào LLM; và (ii) **truy hồi hiệu quả** (efficient retrieval) – giảm thời gian, bộ nhớ và chi phí của bước tìm kiếm mà vẫn giữ chất lượng.

1.2 Bài toán

Cho câu hỏi q và tập $n = 20$ đoạn văn truy hồi được $D = (d_1, \ldots, d_n)$, trong đó đúng một đoạn d_g chứa đáp án (đoạn "vàng"). Một bộ nén C biến (q, D) thành ngữ cảnh ngắn hơn $c = C(q, D)$, sau đó LLM sinh câu trả lời $\hat{a} = \text{LLM}(q, c)$. Mục tiêu là đồng thời:

- tối đa độ chính xác $\text{Acc} = \Pr[\hat{a} \text{ chứa đáp án đúng}]$;
- tối thiểu số token $|c|$ (tức tối đa tỉ lệ nén $|D|/|c|$), độ trễ và chi phí;
- ổn định khi vị trí g của đoạn vàng thay đổi (bền vững với thiên lệch vị trí).

Ở phía truy hồi, cho kho $\mathcal{C}$ gồm khoảng 50 nghìn đoạn văn, ta so sánh các bộ truy hồi theo chất lượng (tỉ lệ top-k chứa đáp án) và hiệu năng (kích thước chỉ mục, thời gian truy vấn).

1.3 Câu hỏi nghiên cứu và đóng góp

Báo cáo trả lời năm câu hỏi:

RQ1. Các bộ nén ngữ cảnh từ thế hệ đầu (LLMLingua-2, LongLLMLingua, 2023–2024) đến thế hệ *pruner* mới (Provence – ICLR 2025, OpenProvence – 2025) giảm được bao nhiêu token mà không làm giảm độ chính xác? Chúng có làm "phẳng" đường cong chữ U không?

RQ2. Các kỹ thuật truy hồi hiệu quả (BM25S, static embedding, Matryoshka, lượng tử hoá int8/binary, HNSW) đánh đổi chất lượng – tốc độ – bộ nhớ thế nào?

RQ3. Kết quả của nhóm so với các công bố trên cùng bộ dữ liệu ra sao?

RQ4. Không có GPU thì pipeline nén + truy hồi có còn dùng được không?

RQ5. Nén mềm "mã hoá một lần" (mỗi tài liệu thành vài vector, tính trước offline) có thay được văn bản khi đọc bằng LLM nhỏ không?

Đóng góp chính của nhóm:

- Xây dựng thư viện mã nguồn mở `effrag` (Python, `uv`) với giao diện thống nhất cho 10 phương pháp nén và 13 cấu hình truy hồi, cùng giao thức đánh giá tái lập được (seed cố định, bộ đếm token chung, cache kết quả LLM).
- Đánh giá có hệ thống trên bộ NQ-Open 20 tài liệu của Liu et al. [28] – bộ dữ liệu mà LongLLMLingua [17] và LLMLingua-2 [33] đã công bố kết quả theo từng vị trí. Theo khảo sát của nhóm, chưa có công bố nào đánh giá họ pruner Provence trên đúng giao thức này.
- Đánh giá nén mềm trên cùng giao thức: PISCO [29] (Mistral-7B) và một bộ nén encode-once do nhóm tự huấn luyện trên Qwen2.5-1.5B, có chế độ lai giữ đoạn quan trọng nhất ở dạng văn bản; với một cổng dừng đặt trước và chẩn đoán chỉ ra vì sao mô hình nhỏ chưa thay được văn bản.
- Một pipeline RAG end-to-end (retriever lượng tử int8 $\rightarrow$ bộ nén $\rightarrow$ reader) trên kho 50,7k đoạn, xác nhận các kết luận của bộ dữ liệu có kiểm soát trong điều kiện truy hồi thật.
- Phân tích đánh đổi độ chính xác – token – độ trễ và một demo tương tác (Gradio) minh hoạ trực quan phần ngữ cảnh mà mỗi phương pháp giữ lại.

Phần còn lại của báo cáo: Mục 2 tổng quan nghiên cứu liên quan; Mục 3 mô tả phương pháp; Mục 4 trình bày cài đặt; Mục 5 phân tích kết quả; Mục 6 kết luận.

2 Nghiên cứu liên quan

Theo khảo sát của Li et al. [25], các phương pháp nén prompt chia thành hai nhóm lớn: *nén cứng* (hard prompt) – đầu ra vẫn là văn bản, dùng được với mọi LLM kể cả qua API; và *nén mềm* (soft prompt) – nén ngữ cảnh thành vector/KV-cache, đòi hỏi can thiệp vào kiến trúc LLM. Bảng 1 tóm tắt các công trình tiêu biểu.

2.1 Nén cứng ở mức token

Selective Context [24] dùng một LM nhỏ (GPT-2) tính lượng thông tin (self-information) của từng cụm từ và xoá các cụm ít thông tin. **LLMLingua** [16] mở rộng ý tưởng với cơ chế phân bổ ngân sách theo phần của prompt và nén lặp theo đoạn (iterative token-level compression) dùng perplexity của một LM nhỏ. **LongLLMLingua** [17] thích nghi LLMLingua cho ngữ cảnh dài và RAG: (i) xếp hạng tài liệu theo perplexity có điều kiện câu hỏi (coarse-grained), (ii) phân bổ tỉ lệ nén động theo thứ hạng, (iii) sắp xếp lại tài liệu để giảm hiện tượng lost-in-the-middle. Trên bộ NQ 20 tài liệu, LongLLMLingua tăng độ chính xác của GPT-3.5-Turbo lên tới 17,1 điểm khi nén 4 lần. **LLMLingua-2** [33] chuyển bài toán thành phân loại token (giữ/bỏ) bằng một bộ mã hoá hai chiều (XLM-RoBERTa-large) được chưng cất từ nhãn do GPT-4 sinh ra trên MeetingBank; nhanh hơn 3–6 lần LLMLingua nhưng *không* dùng câu hỏi (task-agnostic).

2.2 Nén cứng ở mức câu/đoạn (context pruning)

RECOMP [39] huấn luyện bộ nén trích xuất (chọn câu) và bộ nén tóm tắt cho RAG. **CompAct** [40] nén tích cực nhiều vòng bằng LLM 7B; **EXIT** [15] phân loại câu có xét ngữ cảnh bằng Gemma-2B. **Provence** [4] (ICLR 2025) huấn luyện một cross-encoder DeBERTa-v3 vừa *xếp hạng lại* đoạn văn vừa *gắn nhãn câu* cần giữ trong một lượt suy luận, nên chi phí nén gần như bằng không nếu pipeline đã có bước rerank; ngưỡng xác suất quyết định mức nén. Các biến thể mới gồm **XProvence** [32] (đa ngôn ngữ, ECIR 2026) và **OpenProvence** [37] (mã nguồn và dữ liệu mở, giấy phép MIT, backbone ModernBERT). Gần đây, **CORE** [5] (ICML 2026) huấn luyện bộ nén bằng học tăng cường theo chất lượng câu trả lời, và **Sentinel** [42] chọn câu dựa trên attention của một LM 0,5B.

2.3 Nén mềm, bộ nhớ ngữ cảnh và KV-cache

Nén mềm biểu diễn ngữ cảnh bằng *vector* trong không gian embedding của LLM thay cho văn bản. **ICAE** [12] huấn luyện một bộ tự mã hoá: LLM có LoRA đọc ngữ cảnh rồi xuất ra vài "memory slot" mà LLM gốc giải mã được. **xRAG** [2] chiếu embedding truy hồi thành *một* token mềm cho mỗi tài liệu và giữ nguyên LLM. **COCOM** [35] nén mỗi đoạn thành vài *context embedding* và cho thấy việc fine-tune bộ giải mã là then chốt: với tỉ lệ 128×, giữ nguyên bộ giải mã chỉ đạt 0,421 EM trên NQ so với 0,519 khi fine-tune. **PISCO** [29] dùng chính LLM (hai adapter LoRA) làm cả bộ nén lẫn bộ giải mã, huấn luyện bằng chưng cất từ câu trả lời của mô hình thầy; ở tỉ lệ 16× đạt 0,65 so với 0,69 không nén trên NQ (Mistral-7B, 5 đoạn). **OSCAR** [30] chuyển sang nén *có điều kiện câu hỏi* bằng một mô hình 1B, đồng thời xếp hạng lại. **CLaRa** [13] dùng chung biểu diễn nén cho cả truy hồi lẫn sinh. **REFRAG** [27] đưa embedding các khối văn bản thẳng vào bộ giải mã và dùng học tăng cường để quyết định khối nào được "mở" thành văn bản. Các phương pháp này cần dữ liệu và tài nguyên huấn luyện lớn (cụm GPU trung tâm dữ liệu), gắn với một LLM tự host cụ thể và *không* dùng được qua API thương mại.

Bộ chọn lọc và bộ nhớ trong kiến trúc LLM. Các mô hình DeepSeek gần đây tách phần "tìm gì để nhìn" khỏi phần tính toán chính. **DeepSeek-V3.2** [6] thêm *lightning indexer*: một mạng chấm điểm nhỏ có trọng số riêng, được huấn luyện bằng KL theo phân bố attention của mô hình chính, chọn top-k token cho mỗi truy vấn (DeepSeek Sparse Attention). **DeepSeek-V4** [7] kết hợp hai loại attention: một loại nén mỗi nhóm token thành một mục KV rồi để indexer chọn top-k mục, một loại nén mạnh hơn và nhìn toàn bộ. **DeepSeek-V4.1-Flash** [8] thêm cấu trúc encoder–decoder nhân quả, trong đó KV toàn cục của decoder được chiếu từ tầng giữa của encoder. **Engram** [3] là bộ nhớ tham số tra cứu theo n-gram, đóng vai trò "tri thức tĩnh". Ở cấp RAG, hai ý tưởng này tương ứng với (i) bộ chọn lọc rẻ (reranker, Provence) và (ii) bộ nhớ nén cho mỗi tài liệu – chính là hai nhánh mà nhóm kết hợp trong biến thể *lai* ở Mục 3.3.

2.4 Truy hồi hiệu quả

BM25S [31] tính trước điểm BM25 thành ma trận thưa, nhanh hơn hàng trăm lần so với các cài đặt Python truyền thống. Ở phía dense retrieval [19], các mô hình nhúng nhỏ như BGE [38] hay Qwen3-Embedding [41] và EmbeddingGemma [10] hỗ trợ **Matryoshka Representation Learning** [20] – cắt ngắn vector (ví dụ 1024→64 chiều) mà vẫn giữ phần lớn chất lượng. **Lượng tử hoá embedding** [36] (int8 hoặc nhị phân kèm rescoring) giảm 4–32 lần bộ nhớ. **Static embedding** [1] bỏ hẳn Transformer ở bước mã hoá truy vấn, nhanh hơn hàng trăm lần trên CPU. Thư viện **Faiss** [9] cung cấp tìm kiếm chính xác (flat) và xấp xỉ (HNSW, IVF-PQ).

2.5 Bộ dữ liệu và giao thức đánh giá

Liu et al. [28] xây dựng bộ *multi-document QA* từ NaturalQuestions-Open [21]: mỗi câu hỏi kèm 20 đoạn Wikipedia (1 đoạn chứa đáp án do người gán nhãn, 19 đoạn gây nhiễu do Contriever truy hồi), đoạn vàng được đặt ở vị trí 1, 5, 10, 15 hoặc 20. Đây là giao thức mà LongLLMLingua và LLMLingua-2 dùng để báo cáo kết quả theo vị trí, nên chúng tôi chọn làm benchmark chính. Các bộ công cụ như FlashRAG [18] đánh giá bộ nén trên NQ/TriviaQA/HotpotQA với thiết lập khác (top-5, Llama-3-8B, EM).

3 Phương pháp

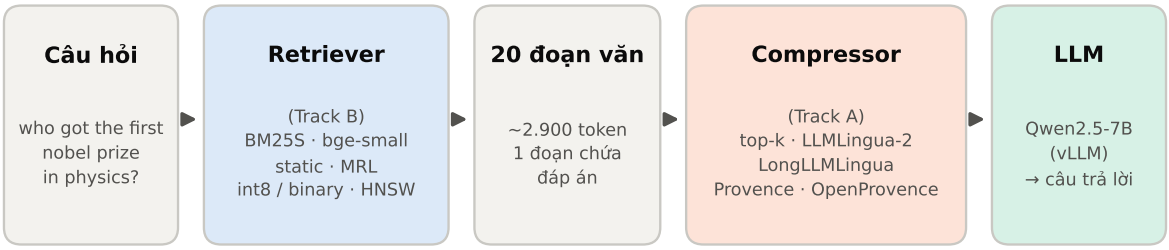
3.1 Kiến trúc tổng thể

Hệ thống gồm ba khối độc lập, ghép nối qua giao diện thống nhất (Hình 1):

1. **Retriever** R: với câu hỏi q, trả về top-k đoạn văn từ kho $\mathcal{C}$.

Bảng 1: Tổng quan các công trình liên quan (H = nén cứng, S = nén mềm, A = kiến trúc attention, R = truy hồi).

Phương pháp	Venue	Loại	Ý tưởng chính	Dùng với API
Selective Context [24]	EMNLP'23	H-token	Self-information từ GPT-2	✓
LLMLingua [16]	EMNLP'23	H-token	Perplexity LM nhỏ, nén lặp	✓
LongLLMLingua [17]	ACL'24	H-token	Có điều kiện câu hỏi, coarse-to-fine	✓
LLMLingua-2 [33]	ACL'24 F	H-token	Phân loại token, chưng cất từ GPT-4	✓
RECOMP [39]	ICLR'24	H-câu	Trích xuất/tóm tắt có huấn luyện	✓
CompAct [40]	EMNLP'24	H-câu	Nén tích cực bằng LLM 7B	✓
EXIT [15]	ACL'25 F	H-câu	Phân loại câu có ngữ cảnh	✓
Provence [4]	ICLR'25	H-câu	Rerank + pruning trong 1 lượt	✓
XProvence [32]	ECIR'26	H-câu	Provence đa ngôn ngữ	✓
CORE [5]	ICML'26	H-câu	Bộ nén huấn luyện bằng RL	✓
xRAG [2]	NeurIPS'24	S	1 token mềm / tài liệu	–
COCOM [35]	WSDM'25	S	Context embedding, decoder fine-tune	–
PISCO [29]	2025	S	LLM vừa nén vừa giải mã (2 LoRA)	–
OSCAR [30]	ICLR'26	S	Nén mềm trực tuyến + rerank	–
REFRAG [27]	2025	S	Embedding khối vào bộ giải mã	–
CLaRa [13]	2025	S	Biểu diễn nén chung cho truy hồi + sinh	–
DeepSeek-V3.2 [6]	2025	A	Lightning indexer chọn top-k token	–
BM25S [31]	2024	R	BM25 thưa, tính trước	✓
MRL [20]	NeurIPS'22	R	Embedding cắt ngắn được	✓
Qwen3-Emb. [41]	2025	R	Embedding 0,6B, MRL 32–1024	✓



Hình 1: Pipeline Efficient RAG: truy hồi → nén ngữ cảnh → sinh câu trả lời. Track A đánh giá khối nén (với 20 đoạn cho sẵn), Track B đánh giá khối truy hồi.

2. **Compressor** C: biến (q, D) thành ngữ cảnh c ngắn hơn. Mọi bộ nén đều trả về *văn bản*, nên dùng được với bất kỳ LLM nào (kể cả qua API).
3. **Generator**: LLM sinh câu trả lời từ prompt chuẩn của Liu et al. [28].

3.2 Các phương pháp nén ngữ cảnh

Chúng tôi chia các bộ nén thành bốn nhóm theo đơn vị bị loại bỏ (Bảng 2).

Tham chiếu. *Closed-book* (không ngữ cảnh) là cận dưới; *Oracle* (chỉ đoạn vàng) là cận trên thực tế; *Full* (giữ nguyên 20 đoạn) là baseline cần vượt qua.

Mức prompt – Truncate. Giữ B token đầu tiên của ngữ cảnh đầy đủ. Đây là cách "ngây thơ" nhất để đạt một ngân sách token; nó loại bỏ hoàn toàn các đoạn ở cuối.

Bảng 2: Các bộ nén được đánh giá. "Hỏi" = có dùng câu hỏi khi nén.

Nhóm	Phương pháp	Mô hình	Hỏi	Tham số
Tham chiếu	Closed-book / Oracle / Full	–	–	–
Prompt	Truncate	–	–	0
Tài liệu	BM25 top-k	BM25S	✓	0
	Dense top-k	bge-small-en-v1.5	✓	33M
	Rerank top-k	ms-marco-MiniLM-L6-v2	✓	23M
Token	LLMLingua-2	xlm-roberta-large	–	559M
	LongLLMLingua	Qwen2.5-0.5B-Instruct	✓	494M
	Rerank → LLMLingua-2	MiniLM + XLM-R	✓	582M
Câu	Provence	deberta-v3-large	✓	435M
	OpenProvence	gte-modernbert-base	✓	150M

Mức tài liệu – Top-k selection. Chấm điểm từng đoạn theo câu hỏi rồi giữ k đoạn có điểm cao nhất, *giữ nguyên thứ tự ban đầu*. Ba bộ chấm điểm: BM25 (thưa, từ vựng), bi-encoder `bge-small-en-v1.5` [38] (cosine), và cross-encoder `ms-marco-MiniLM-L6-v2` (reranker). Với 20 đoạn trung bình ∼2.900 token, $k = 10$ và $k = 5$ tương ứng nén khoảng $2\times$ và $4\times$, khớp với hai mức ngân sách trong bảng của LongLLMLingua.

Mức token. **LLMLingua-2** [33]: bộ phân loại token XLM-RoBERTa-large dự đoán xác suất giữ $p(\text{keep} \mid x_i)$; giữ các token có xác suất cao nhất sao cho đạt tỉ lệ r (ngưỡng toàn cục trên cả 20 đoạn). **LongLLMLingua** [17]: tính perplexity có điều kiện câu hỏi bằng LM nhân quả nhỏ (Qwen2.5-0.5B-Instruct thay cho Llama-2-7B trong bài gốc) để xếp hạng đoạn và nén token với ngân sách B token. Ngoài ra, chúng tôi ghép **reranker top-10 → LLMLingua-2** ($r = 0{,}5$) để mô phỏng chiến lược coarse-to-fine.

Mức câu – Context pruning. **Provence** [4] là một cross-encoder DeBERTa-v3-large với hai đầu ra: điểm liên quan của cả đoạn (reranking) và xác suất giữ của từng token, được gộp thành quyết định giữ/bỏ từng *câu* theo ngưỡng τ. Ngưỡng nhỏ giữ nhiều câu hơn. Đoạn không còn câu nào bị loại hẳn. **OpenProvence** [37] dùng cùng ý tưởng với backbone ModernBERT-base (150M tham số, nhỏ hơn 3 lần). Chúng tôi quét $\tau \in \{0{,}01; 0{,}1; 0{,}5\}$ cho Provence và $\tau \in \{0{,}1; 0{,}5\}$ cho OpenProvence.

Tính bất biến theo thứ tự. Các bộ nén mức tài liệu, mức câu và LLMLingua-2 xử lý từng đoạn độc lập với vị trí của nó. Chúng tôi tận dụng tính chất này: kết quả nén của một tập đoạn được ghi nhớ và tái sử dụng cho mọi vị trí đoạn vàng (chỉ thứ tự thay đổi), giảm 3 lần thời gian nén; độ trễ báo cáo là độ trễ đo ở lần tính đầu tiên. Sau khi nén, các đoạn còn lại *giữ nguyên thứ tự* và được đánh số lại, nhờ đó biến "vị trí đoạn vàng" vẫn có ý nghĩa, giống thiết lập của các bảng đã công bố.

3.3 Nén mềm "encode-once" (phương pháp đề xuất)

Các bộ nén ở trên đều giữ ngữ cảnh dưới dạng *văn bản*, nên tỉ lệ nén bị chặn bởi lượng chữ phải giữ lại. Chúng tôi thử một hướng khác: nén mỗi đoạn thành một *ma trận* nhỏ gồm m vector trong không gian embedding của LLM, theo dòng PISCO/COCOM [29, 35], và huấn luyện với ngân sách của một GPU 12 GB.

Kiến trúc. Một LLM nhân quả f_θ (Qwen2.5-1.5B-Instruct) đóng hai vai trò nhờ hai adapter LoRA [14] riêng:

- **Encoder** E (adapter ϕ_E): đọc đoạn d_j (tối đa 128 token, gồm tiêu đề và nội dung), nối thêm $m = 8$ vector nhớ học được $\mathbf{s}_1, \ldots, \mathbf{s}_m$. Trạng thái ẩn cuối tại các vị trí nhớ được bỏ đi $c = 8$ kênh *massive*

activation, chiếu tuyến tính, chuẩn hoá L2 và nhân với hệ số học được, ra $M_j = E(d_j) \in \mathbb{R}^{m \times h}$ (xem đoạn "Khởi tạo" bên dưới).

- **Decoder** D (adapter ϕ_D): nhận *đúng* prompt Lost-in-the-Middle (cùng chat template), trong đó phần nội dung của mỗi `Document [i](Title: ...)` được thay bằng m vector M_j, rồi sinh câu trả lời.

Vì E không nhìn câu hỏi, M_j có thể **tính trước một lần cho cả kho** (encode-once) và lưu lại như một chỉ mục. Khi hỏi, decoder chỉ xử lý khoảng $20 \times (m + \text{token tiêu đề})$ vị trí thay vì 2.874 token: trung bình 416 vị trí ($m = 8$, 500 câu), tức tỉ lệ nén ngữ cảnh 6,9× nếu tính cả tiêu đề và 16,4× nếu chỉ tính phần nội dung.

Khởi tạo. Trạng thái ẩn cuối của Qwen2.5 lớn hơn embedding token hàng trăm lần và tập trung năng lượng vào vài kênh cố định (hiện tượng *massive activation*): ở Qwen2.5-1.5B, kênh lớn nhất chiếm trung bình 28% bình phương chuẩn của vector nhớ. Vì lớp đầu tiên của decoder chuẩn hoá RMS từng vị trí, chỉ *hướng* của vector nhớ có ý nghĩa: chuẩn hoá L2 hay thu nhỏ vector đều không giúp gì. Đo trên 6 mẫu huấn luyện (4 mẫu 5 đoạn, 2 mẫu 20 đoạn) khi chưa huấn luyện, trong cùng một lần khởi tạo, loss trên câu trả lời là 15,5 nat (tệ hơn đoán đều trên từ vựng, $\ln |V| \approx 11{,}9$; closed-book 5,6), tức các vector nhớ làm decoder "lạc" hoàn toàn. Bỏ 8 kênh có biên độ trung bình lớn nhất (xác định một lần trên một câu hiệu chuẩn, lưu cùng adapter) đưa loss về 7,2, gần mức khi thay toàn bộ vector nhớ bằng 0 (7,0); quá trình huấn luyện khi đó chỉ cần dạy vector nhớ mang thông tin thay vì trước hết phải "gỡ" nhiễu. Ngoài ra, khi bật gradient checkpointing, encoder bị tính lại trong lúc lan truyền ngược sau khi adapter decoder đã được kích hoạt; chúng tôi chuyển lại adapter encoder bằng một hook trên trạng thái ẩn của encoder (không sửa, gradient encoder có cosine $-0{,}01$ so với tính không checkpoint).

Biến thể lai (cứng + mềm). Lấy cảm hứng từ việc DeepSeek-V4 kết hợp "chọn top-k mục để nhìn chi tiết" với "nhìn toàn bộ ở dạng nén" [7], biến thể *hybrid* giữ nguyên văn bản của đoạn được cross-encoder xếp hạng cao nhất, các đoạn còn lại ở dạng vector nhớ. Lúc huấn luyện, với xác suất 0,5 một đoạn được "mở" thành văn bản (là đoạn vàng với xác suất 0,6), để decoder quen với ngữ cảnh trộn và không mặc định rằng đoạn mở luôn chứa đáp án.

Huấn luyện. Hàm mất mát là cross-entropy trên token câu trả lời; ϕ_E, ϕ_D, $\mathbf{s}$ và phép chiếu được học đồng thời end-to-end, phần thân LLM được đóng băng.

- **Dữ liệu**: 15.000 câu hỏi từ NQ-train của Tevatron [11] (đoạn Wikipedia 100 từ kiểu DPR, cùng nguồn với benchmark), đã loại 33 câu trùng với 2.655 câu của tập kiểm tra LITM. Mỗi mẫu có $k \in \{5, 10, 20\}$ đoạn (1 đoạn đúng và các đoạn nhiễu khó do BM25 tìm), với đoạn đúng đặt ở *vị trí ngẫu nhiên*.
- **Siêu tham số**: LoRA $r = 16$ trên mọi lớp tuyến tính, AdamW với lr 10^{-4}, warm-up 50 bước rồi cosine, 8 mẫu mỗi bước, gradient checkpointing, bf16, 1 epoch.

Cổng dừng. Trước khi đánh giá trên LITM, mô hình phải vượt một cổng trên 200 câu dev (Tevatron, 20 đoạn). Điều kiện: độ chính xác cao hơn closed-book của cùng LLM ít nhất 10 điểm *và* đạt ít nhất 70% độ chính xác khi đọc văn bản đầy đủ. Cổng này được đặt trước để tránh "chọn kết quả đẹp" sau khi đã thấy số liệu kiểm tra.

Đối chứng với checkpoint công bố. Song song, chúng tôi chạy `naver/pisco-mistral` (Mistral-7B, lượng tử 4-bit) trên cùng 500 câu và 3 vị trí, kèm các dòng full-text, closed-book và oracle của chính bộ giải mã đó. Mục đích là tách ảnh hưởng của dữ liệu và quy mô huấn luyện khỏi ảnh hưởng của phương pháp.

3.4 Các phương pháp truy hồi hiệu quả

Trên kho gộp ∼50,7 nghìn đoạn, chúng tôi so sánh:

- **BM25S** [31] với stemming và loại stopword.
- **Static embedding** `static-retrieval-mrl-en-v1` [1] (không có Transformer, 1024 chiều, hỗ trợ Matryoshka).
- **bge-small-en-v1.5** (33M, 384 chiều) với chỉ mục chính xác (flat), xấp xỉ (HNSW, $M = 32$), lượng tử int8 và nhị phân.

Cắt Matryoshka được minh hoạ qua static embedding 1024 → 256 chiều. Cấu hình cho Qwen3-Embedding-0.6B [41] có sẵn trong mã (`run_retrieval.py`) nhưng không được chạy trong báo cáo: mã hoá 50,7k đoạn bằng mô hình 0,6B chiếm GPU trong khi GPU đang phục vụ LLM, và lợi ích của nó đã được công bố trên MTEB. Lượng tử int8 ánh xạ tuyến tính mỗi chiều về 256 mức (theo min/max trên kho). Lượng tử nhị phân giữ dấu của từng chiều (1 bit), tìm kiếm bằng khoảng cách Hamming trên $4k$ ứng viên, sau đó *rescore* bằng tích vô hướng giữa vector truy vấn float và vector tài liệu $\{-1, +1\}$ [36] – không cần lưu vector float của kho.

3.5 Giao thức đánh giá

Dữ liệu. 500 câu hỏi lấy ngẫu nhiên (seed 42) từ 2.655 câu của NQ-Open 20 tài liệu; cùng 500 câu cho ba vị trí đoạn vàng **1, 10, 20** – đúng ba vị trí mà LongLLMLingua và LLMLingua-2 công bố số liệu, nên mọi dòng của chúng tôi đều có điểm đối chiếu. Hai vị trí 5 và 15 bị bỏ vì chỉ làm mịn đường cong mà không có số công bố để so sánh, trong khi tăng 67% chi phí. Tổng cộng 1.500 prompt cho mỗi cấu hình; các tham chiếu không phụ thuộc vị trí (closed-book, oracle) chỉ chạy một lần.

Rút gọn ma trận thí nghiệm. Thay vì chạy mọi tổ hợp, chúng tôi (i) chạy *dry-run* miễn phí cho mọi bộ nén để đo tỉ lệ nén và context recall, rồi chỉ gọi LLM cho các cấu hình đại diện; (ii) với BM25/Dense top-5 – vốn có số công bố và recall thấp rõ rệt – chỉ chạy ở vị trí 10; (iii) LongLLMLingua (chậm nhất, ≈0,6 s/câu) chỉ chạy 200 câu ở vị trí 10; (iv) dùng lại số liệu GPT-3.5 đã công bố làm tham chiếu thay vì tái lập.

Prompt. Dùng nguyên mẫu prompt của Liu et al. [28]:

```
Write a high-quality answer for the given question using only the
   provided search
results (some of which might be irrelevant).

Document [1](Title: ...) ...
...
Question: {question}
Answer:
```

Chỉ số.

- **Accuracy**: câu trả lời đúng nếu một trong các đáp án chuẩn (sau chuẩn hoá chữ thường, bỏ dấu câu và mạo từ) xuất hiện trong output – định nghĩa của Liu et al. [28], cũng được dùng bởi LongLLMLingua và LLMLingua-2.
- **Context recall**: đáp án còn xuất hiện trong ngữ cảnh sau nén hay không – đo khả năng "giữ bằng chứng" độc lập với LLM.

- **Token và tỉ lệ nén**: đếm bằng một bộ tách từ chung (tiktoken `o200k_base`) cho mọi phương pháp; tỉ lệ nén = token gốc / token sau nén.
- **Độ trễ**: thời gian nén mỗi câu hỏi và thời gian gọi LLM.
- Truy hồi: **answer-hit@**k (top-k chứa đáp án, như DPR [19]), **MRR@10** của đoạn vàng, kích thước chỉ mục, thời gian truy vấn.

4 Cài đặt hệ thống

4.1 Tổ chức mã nguồn

Toàn bộ mã nguồn được công khai tại `https://github.com/hotamago/nlp-efficient-rag`, viết bằng Python 3.11 và quản lý môi trường bằng `uv` (khoá phiên bản trong `uv.lock`). Gói `effrag` gồm các mô-đun:

- `data.py`: đọc bộ NQ-Open 20 tài liệu, lấy mẫu cố định theo seed, gộp kho truy hồi.
- `compression/`: lớp trừu tượng `Compressor` và `DocwiseCompressor` (ghi nhớ kết quả theo tập tài liệu), các bộ nén ở Bảng 2 và bộ ghép `Chain` (nén nhiều tầng).
- `retrieval.py`: `BM25Retriever` và `DenseRetriever` (cắt Matryoshka, lượng tử int8/nhị phân, chỉ mục flat/HNSW).
- `generation.py`: client tương thích OpenAI, có cache theo hash của prompt và thử lại khi lỗi; cùng một mã chạy được với API thương mại, vLLM hoặc Ollama.
- `metrics.py`, `tokens.py`, `results.py`: chỉ số, bộ đếm token chung và lược đồ kết quả (CSV/JSONL) dùng chung cho cả nhóm.
- `scripts/`: chạy thí nghiệm (`run_compression.py`, `run_retrieval.py`, `run_all_compression.sh`), vẽ hình (`make_figures.py`); `app/demo.py`: demo Gradio.

Thêm một bộ nén mới chỉ cần kế thừa `Compressor` và cài đặt một hàm:

```
class MyPruner(DocwiseCompressor):
    name = "my_pruner"
    def _compress_docs(self, question, docs):    # -> one string per
    document ("" = drop)
        return [prune(question, d) for d in docs]
```

4.2 Thư viện, mô hình và dữ liệu sử dụng

4.3 Thông số thực nghiệm

- **Phần cứng**: 1 GPU NVIDIA RTX 4070 SUPER 12GB, CPU 20 luồng (mặc định 4 luồng cho mỗi tiến trình, 8 luồng khi đo độ trễ CPU), 24GB RAM, WSL2.
- **Sinh câu trả lời**: Qwen2.5-7B-Instruct (lượng tử AWQ 4-bit), `temperature`=0, tối đa 100 token sinh (như Liu et al. [28]), độ dài ngữ cảnh tối đa 4.096 token.
- **Lấy mẫu**: 500 câu hỏi, seed 42, 3 vị trí đoạn vàng $\{1, 10, 20\}$.
- **Ngân sách nén**: tương ứng $2\times$ ($\approx$1.450 token) và $4\times$ ($\approx$730 token) so với $\approx$2.900 token của ngữ cảnh đầy đủ – cùng hai mức của bảng kết quả LongLLMLingua.

Bảng 3: Thành phần có sẵn được sử dụng.

Thành phần	Phiên bản / nguồn
Dữ liệu	NQ-Open multi-document QA, 20 tài liệu [28] (`nelson-liu/lost-in-the-middle`)
LLM sinh câu trả lời	`Qwen/Qwen2.5-7B-Instruct-AWQ` [26, 34], phục vụ bằng vLLM 0.31 [22]
Nén token	`llmlingua` 0.2.2; `microsoft/llmlingua-2-xlm-roberta-large-meetingbank`; `Qwen/Qwen2.5-0.5B-Instruct`
Nén câu	`naver/provence-reranker-debertav3-v1`; `hotchpotch/open-provence-reranker-v1-gte-modernbert-base`
Truy hồi	`bm25s` 0.3, `faiss-cpu` 1.15, `sentence-transformers` 5.7; `BAAI/bge-small-en-v1.5`, `static-retrieval-mrl-en-v1`, `cross-encoder/ms-marco-MiniLM-L6-v2`
Nền tảng	PyTorch 2.14, Transformers 4.57, tiktoken 0.14, Gradio 6

- **LongLLMLingua**: `condition_compare=True`, `condition_in_question="after"`, `rank_method="longllmlingua"`, `dynamic_context_compression_ratio=0.4`, `context_budget="+100"`, `reorder_context="original"`.
- **LLMLingua-2**: `rate`$\in \{0{,}5; 0{,}25\}$, giữ bắt buộc ký tự xuống dòng, "?" và ".".

Điều chỉnh tương thích. `llmlingua` 0.2.2 thao tác KV-cache dạng tuple cũ; chúng tôi bọc hàm `forward` của LM để chuyển đổi qua lại với `DynamicCache` của Transformers mới. Các mô hình Provence dùng mã tuỳ biến (`trust_remote_code`); với OpenProvence, mỗi cặp (câu hỏi, đoạn) được đưa vào như một phần tử độc lập của batch.

4.4 Demo

`app/demo.py` là ứng dụng Gradio chỉ gọi API của gói `effrag` (không sao chép logic), gồm hai tab:

- **So sánh bộ nén**: chọn một câu hỏi trong 200 câu đầu, vị trí đoạn vàng và các bộ nén; ứng dụng hiển thị số token trước/sau nén, thời gian nén, đáp án còn trong ngữ cảnh hay không và câu trả lời của LLM (Hình 2). Với bộ nén theo đoạn, từng câu được giữ được tô xanh, câu bị loại bị gạch, đoạn vàng được đánh dấu (Hình 3).
- **RAG end-to-end**: câu hỏi tự do $\rightarrow$ BM25S hoặc bge-small trên kho 50,7k đoạn $\rightarrow$ bộ nén $\rightarrow$ LLM.

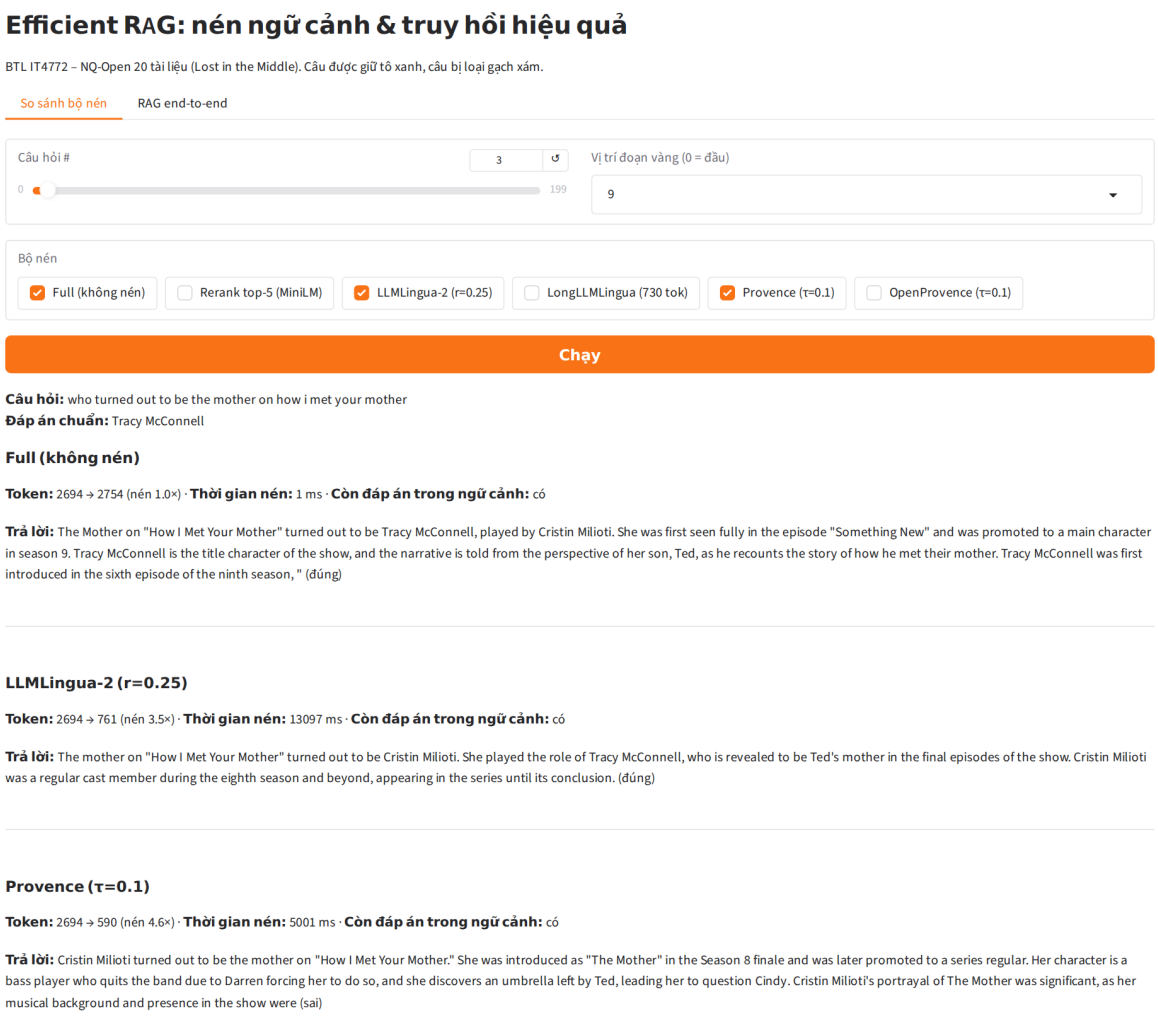
Khi không có máy chủ LLM, demo vẫn chạy phần truy hồi và nén; mọi mô hình nén chạy được trên CPU (`EFFRAG_DEVICE=cpu`). Ảnh chụp dưới đây được lấy khi chạy toàn bộ bộ nén trên CPU (4 luồng, máy đang chạy job khác), LLM qua vLLM.

5 Kết quả và thảo luận

Mọi số liệu trong mục này được sinh tự động từ `results/` bằng `scripts/make_figures.py`; mỗi câu hỏi có một dòng trong các tệp `.jsonl` đã commit. Trừ khi ghi chú khác, $n = 500$ câu hỏi, các vị trí đoạn vàng 1/10/20, reader Qwen2.5-7B-Instruct-AWQ.

5.1 Bộ nén giữ được bao nhiêu bằng chứng?

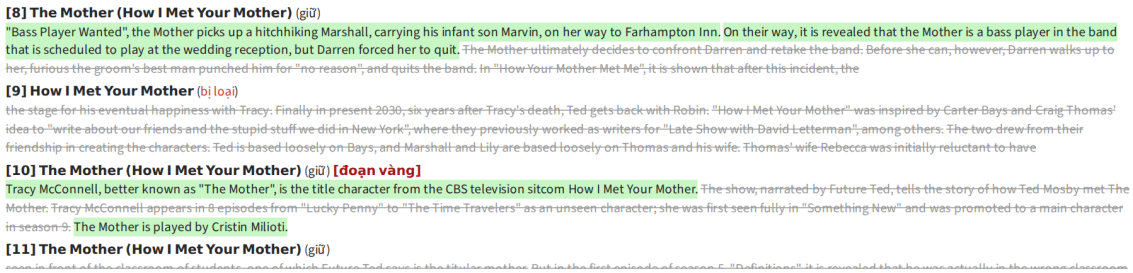
Bảng 4 đo *context recall* – tỉ lệ câu hỏi mà đáp án còn nằm trong ngữ cảnh sau nén – cho mọi bộ nén mà không cần gọi LLM. Đây là trần trên của độ chính xác sau nén: nếu đáp án đã bị xoá thì reader chỉ còn trả lời được bằng tri thức tham số (closed-book).



Hình 2: Tab "So sánh bộ nén" của demo (câu hỏi #3, đoạn vàng ở vị trí 10, chạy trên CPU). LLMLingua-2 và Provence đều giữ được đáp án; với Provence, reader 7B trả lời bằng tên diễn viên (Cristin Milioti) thay vì tên nhân vật – một kiểu lỗi được phân tích ở Mục 5.9.

Nhận xét.

- **Ở cùng mức nén ≈4×, nén mức câu giữ bằng chứng tốt nhất**: Provence ($\tau = 0{,}1$) giữ 91,6% đáp án với 621 token (4,6×), hơn Rerank top-5 (79,4%) 12,2 điểm, hơn LongLLMLingua (77,5%) và LLMLingua-2 $r = 0{,}25$ (68,0%) lần lượt 14 và 24 điểm. Cắt tỉa theo *câu*, có điều kiện câu hỏi, giữ nguyên các câu chứa đáp án; còn bỏ token rời rạc (LLMLingua-2 không nhìn câu hỏi) có thể làm mất chính thực thể cần trả lời.
- **Ở ≈2×**, OpenProvence (96,4%, 2,6×) và Provence $\tau = 0{,}01$ (95,8%, 2,5×) gần như không mất bằng chứng; Rerank top-10 (89,8%) và LLMLingua-2 $r = 0{,}5$ (88,2%) mất khoảng 10%.
- **BM25 kém trong bối cảnh này** (53,2% ở top-5): 19 đoạn nhiễu của bộ dữ liệu vốn được Contriever truy hồi là "hard negative" có độ trùng từ vựng cao với câu hỏi. Cross-encoder MiniLM (22M tham số), đọc câu hỏi và đoạn cùng lúc, cho recall cao hơn bi-encoder bge-small 10,4 điểm mà vẫn nhanh hơn (14 so với 121 ms mỗi câu trong cài đặt của chúng tôi).
- **Truncate** đúng như kỳ vọng: giữ trọn đáp án khi đoạn vàng ở đầu và mất gần hết khi đoạn vàng ở giữa/cuối – một baseline cho thấy vì sao cần nén *có điều kiện câu hỏi*.



Hình 3: Ngữ cảnh sau nén của Provence trong demo: câu được giữ (nền xanh), câu bị loại (gạch ngang), đoạn bị loại hoàn toàn và đoạn vàng.

Bảng 4: Context recall (%) theo vị trí đoạn vàng, số token sau nén, tỉ lệ nén và độ trễ nén trung bình mỗi câu hỏi (GPU). Bộ nén mức tài liệu/token/câu xử lý từng đoạn độc lập nên recall không phụ thuộc vị trí. LongLLMLingua: $n = 200$, chỉ vị trí 10.

Bộ nén	Token	Nén	R@1	R@10	R@20	ms/câu
Truncate 1450	1450	2.0×	100.0	92.6	20.6	0
Truncate 730	730	3.9×	100.0	16.0	16.0	0
BM25 top-10	1426	2.0×	75.4	75.4	75.4	2
Dense top-10	1425	2.0×	81.8	81.8	81.8	196
Rerank top-10	1425	2.0×	89.8	89.8	89.8	14
BM25 top-5	705	4.1×	53.2	53.2	53.2	2
Dense top-5	702	4.1×	69.0	69.0	69.0	121
Rerank top-5	701	4.1×	79.4	79.4	79.4	14
LLMLingua-2 r=0.5	1500	1.9×	88.2	88.2	88.2	462
LLMLingua-2 r=0.25	827	3.5×	68.0	68.0	68.0	462
Rerank-10 → LLMLingua-2	743	3.9×	79.2	79.2	79.2	253
LongLLMLingua 730t	674	4.3×	–	77.5	–	620
Provence τ=0.01	1136	2.5×	95.8	95.8	95.8	212
OpenProvence τ=0.1	1091	2.6×	96.4	96.4	96.4	455
Provence τ=0.1	621	4.6×	91.6	91.6	91.6	210

5.2 Độ chính xác của RAG sau nén

Bảng 5 và Hình 4 cho thấy khoảng cách giữa closed-book (38,8%) và oracle (92,6%) là 53,8 điểm: ngữ cảnh rất quan trọng, nhưng 20 đoạn đầy đủ (65,3%) chỉ khai thác được một nửa khoảng cách đó vì nhiễu. Kiểm định bootstrap ghép cặp (10.000 lần lấy mẫu, khoảng tin cậy 95%, `results/compression/significance.csv`) so với ngữ cảnh đầy đủ cho các kết luận sau.

- **Ở 4×, nén mức câu gần như không mất độ chính xác.** Provence $\tau = 0{,}1$ dùng 621 token (4,6×) và đạt 64,0% so với 65,3%; chênh lệch ở cả ba vị trí đều không có ý nghĩa thống kê ($-1{,}8$ [$-5{,}6$; $2{,}0$], $+0{,}4$ [$-3{,}4$; $4{,}2$], $-2{,}4$ [$-6{,}2$; $1{,}4$] điểm ở vị trí 1/10/20). Đây là cấu hình tốt nhất trên toàn bộ đường Pareto.
- **Chọn tài liệu bằng cross-encoder đứng thứ hai.** Rerank top-5 (4,1×) đạt 62,7%, nhưng giảm có ý nghĩa ở vị trí 20 ($-6{,}0$ [$-10{,}0$; $-2{,}2$]) vì 20,6% câu mất đoạn chứa đáp án. Rerank top-10 (2×) đạt 63,7%. Bộ chọn rẻ hơn kém xa: Dense top-5 56,0%, BM25 top-5 45,4% (vị trí 10).
- **Nén mức token thất bại với reader 7B.** LLMLingua-2 $r = 0{,}25$ (3,5×) chỉ đạt 38,3% – ngang closed-book – dù context recall vẫn còn 68%; ở $r = 0{,}5$ (1,9×) cũng chỉ 53,5%. Ghép rerank-10 trước LLMLingua-2 (cấu hình tốt nhất của LLMLingua-2 trên GPT-3.5) đạt 53,4%, giảm có ý nghĩa ở mọi vị trí. Mục 5.9 cho thấy phần lớn thiệt hại không đến từ việc xoá đáp án mà từ việc văn bản bị xé vụn khiến reader đọc sai.

Bảng 5: Kết quả chính (Qwen2.5-7B-Instruct-AWQ, $n = 500$): accuracy (%) theo vị trí đoạn vàng và trung bình (TB), số token ngữ cảnh, tỉ lệ nén, context recall (%) và độ trễ nén (ms/câu, GPU). BM25 và Dense top-5 chỉ chạy ở vị trí 10 (đã có số công bố, Mục 5.4); LongLLMLingua ($n = 200$) ở Bảng 6. In đậm: TB cao nhất trong các bộ nén.

Cấu hình	@1	@10	@20	TB	Token	Nén	Recall	ms
Tham chiếu								
Full (20 docs)	65.6	62.2	68.0	65.3	2875	1.0×	100.0	0
Oracle	92.6	92.6	92.6	92.6	113	25.5×	100.0	0
Closed-book	38.8	38.8	38.8	38.8	0	–	0.2	0
Prompt								
Truncate 730	74.0	19.6	19.6	37.7	730	3.9×	44.0	0
Tài liệu								
Rerank top-10	65.0	62.2	64.0	63.7	1425	2.0×	89.8	14
BM25 top-5	–	45.4	–	–	705	4.1×	53.2	2
Dense top-5	–	56.0	–	–	702	4.1×	69.0	121
Rerank top-5	64.2	62.0	62.0	62.7	701	4.1×	79.4	14
Token								
LLMLingua-2 r=0.5	56.0	52.6	52.0	53.5	1500	1.9×	88.2	462
LLMLingua-2 r=0.25	36.6	40.8	37.6	38.3	827	3.5×	68.0	462
Rerank-10 → LLMLingua-2	54.0	53.8	52.4	53.4	743	3.9×	79.2	253
Câu								
Provence τ=0.01	64.4	61.0	63.2	62.9	1136	2.5×	95.8	212
OpenProvence τ=0.1	63.0	61.4	62.0	62.1	1091	2.6×	96.4	455
Provence τ=0.1	63.8	62.6	65.6	**64.0**	621	4.6×	91.6	210

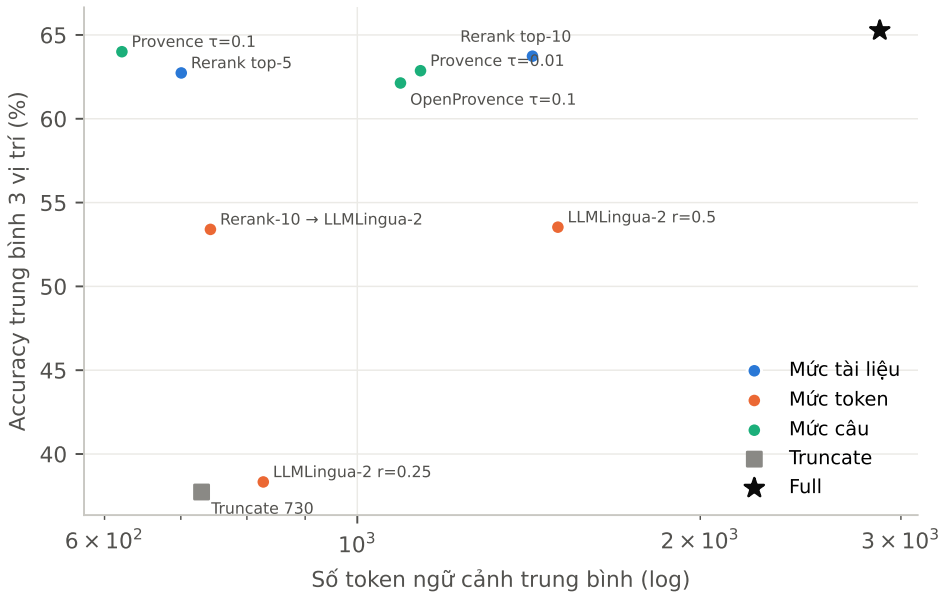
- **OpenProvence** (2,6×) giữ bằng chứng tốt nhất (recall 96,4%) nhưng accuracy 62,1%, thấp hơn Provence $\tau = 0{,}1$ dù giữ nhiều token hơn: giữ thêm câu nhiễu không giúp reader.
- **Truncate** là trường hợp đặc biệt: khi đoạn vàng ở đầu, cắt còn 730 token lại *tăng* accuracy lên 74,0% (+8,4 [5,0; 12,0]) vì loại bỏ 15 đoạn nhiễu; nhưng ở vị trí 10/20 chỉ còn 19,6%. Điều này xác nhận rằng ngữ cảnh ngắn và đúng tốt hơn ngữ cảnh dài – vấn đề là chọn *đúng* phần nào.

Chi phí. Với vLLM, thời gian prefill tỉ lệ với số token: Provence giảm 2.875 → 621 token mỗi câu, đổi lại khoảng 200 ms nén trên GPU (4,9 s trên CPU, Bảng 11). Rerank top-5 rẻ hơn hẳn (16 ms GPU, 196 ms CPU) với độ chính xác thấp hơn 1,3 điểm – là lựa chọn hợp lý khi không có GPU.

5.3 Nén và hiện tượng *lost in the middle*

Với GPT-3.5-Turbo, Liu et al. [28] và LongLLMLingua báo cáo đường cong chữ U sâu: 75,7 / 54,1 / 63,1% ở vị trí 1/10/20, chênh 21,6 điểm giữa đầu và giữa. Với Qwen2.5-7B-Instruct, đường cong của ngữ cảnh đầy đủ gần như phẳng (65,6 / 62,2 / 68,0%): chênh lệch lớn nhất chỉ 5,8 điểm, và vị trí cuối còn tốt hơn vị trí đầu (Hình 5). Các mô hình mới được huấn luyện với ngữ cảnh dài hơn nhiều (Qwen2.5: 32K–128K token) nên 2.900 token không còn là ngữ cảnh "dài".

Hệ quả là nén *không có nhiều chỗ để làm phẳng* đường cong: Provence $\tau = 0{,}1$ cho 63,8 / 62,6 / 65,6%, gần như song song với ngữ cảnh đầy đủ. Các bộ nén không phụ thuộc vị trí (mức tài liệu, câu, token) giữ nguyên hình dạng đường cong của reader; chỉ truncate – vốn phụ thuộc vị trí – tạo ra độ dốc lớn.



Hình 4: Accuracy trung bình 3 vị trí theo số token ngữ cảnh (thang log). Góc trên bên trái là tốt nhất: ít token, độ chính xác cao.

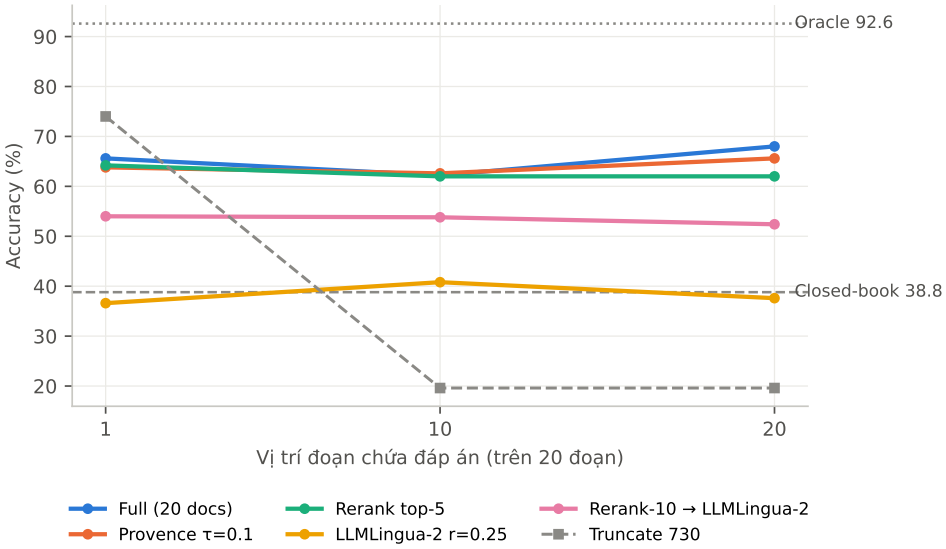
Bảng 6: So sánh với số liệu đã công bố trên cùng benchmark (GPT-3.5-Turbo-0613; LongLLMLingua [17] Bảng 1 và LLMLingua-2 [33] Bảng 4). Hàng đầu: accuracy tuyệt đối (%) của ngữ cảnh đầy đủ; các hàng sau: *chênh lệch* (điểm) so với ngữ cảnh đầy đủ của chính hệ đó. LongLLMLingua của chúng tôi: so cặp trên 193 câu chung với lần chạy đầy đủ.

GPT-3.5-Turbo (đã công bố)				Qwen2.5-7B-AWQ (của chúng tôi)			
Phương pháp	@1	@10	@20	Cấu hình tương ứng	@1	@10	@20
Original prompt	75.7	54.1	63.1	Full (20 docs)	65.6	62.2	68.0
BM25 (4×)	-35.1	-15.9	-26.5	BM25 top-5	–	-16.8	–
SBERT (4×)	-8.8	+4.9	-2.8	Dense top-5	–	-6.2	–
LLMLingua-2 (4×)	-27.1	-10.5	-23.2	LLMLingua-2 r=0.25	-29.0	-21.4	-30.4
LLMLingua-2 + lọc tài liệu (4×)	-1.7	+12.9	+2.2	Rerank-10 → LLMLingua-2	-11.6	-8.4	-15.6
LongLLMLingua (4×)	-0.7	+17.1	+11.6	LongLLMLingua 730t	–	+1.0	–

5.4 Đối chiếu với số liệu đã công bố

Vì GPT-3.5-Turbo-0613 đã ngừng hoạt động, chúng tôi so sánh *độ chênh so với baseline của chính mỗi hệ* (Bảng 6) thay vì số tuyệt đối.

- **Tái lập được**: thứ hạng giữa các bộ chọn tài liệu (SBERT/Dense tốt hơn BM25 rõ rệt) và mức sụt nặng của LLMLingua-2 đơn lẻ ở 4× ($-29{,}0$ / $-21{,}4$ / $-30{,}4$ điểm so với $-27{,}1$ / $-10{,}5$ / $-23{,}2$ được công bố).
- **Không tái lập được**: kết luận "nén làm *tăng* độ chính xác ở giữa" – LongLLMLingua $+17{,}1$ và LLMLingua-2 + lọc tài liệu $+12{,}9$ điểm ở vị trí 10. Với reader của chúng tôi, mức thay đổi tương ứng là $+1{,}0$ (không có ý nghĩa thống kê, khoảng tin cậy $[-6{,}7;\ 8{,}8]$) và $-8{,}4$. Lời giải thích hợp lý nhất là ở Mục 5.3: lợi ích chính của các phương pháp này trên GPT-3.5 là *khắc phục lost in the middle*, mà reader hiện đại hầu như không còn mắc.
- Về vị trí đầu và cuối, cả hai hệ đều cho thấy nén 4× ở mức tốt nhất chỉ *hoà* với ngữ cảnh đầy đủ (LongLLMLingua $-0{,}7$ trên GPT-3.5; Provence $-1{,}8$ / $-2{,}4$ của chúng tôi).



Hình 5: Accuracy theo vị trí đoạn chứa đáp án. Đường ngang: closed-book và oracle.

5.5 Nén mềm

PISCO. Bảng 7 so sánh PISCO với chính reader nền của nó (Mistral-7B-Instruct-v0.2, lượng tử 4-bit nf4) đọc 20 đoạn dạng văn bản, cùng prompt. PISCO thay 2.875 token văn bản bằng 160 vector nhớ (8 vector/đoạn, nén 18× theo bộ đếm token chung) và vẫn đạt 48,2% accuracy trung bình, cao hơn 12 điểm so với closed-book của chính reader này (36,2%) nhưng còn xa oracle (89,6%). Tuy vậy, so với văn bản đầy đủ trên cùng reader (60,3%), PISCO giảm 8,8 / 10,4 / 17,2 điểm ở vị trí 1 / 10 / 20, cả ba đều có ý nghĩa thống kê (khoảng tin cậy bootstrap 95% lần lượt $[-13,6;\ -4,0]$, $[-14,8;\ -6,0]$, $[-22,0;\ -12,6]$). Mức giảm lớn nhất khi đoạn vàng nằm cuối: đường cong vị trí của PISCO có dạng chữ U ngược (52,8 ở giữa, 44,0 ở cuối), trong khi văn bản dao động ít hơn (56,6–63,2). Như vậy, trên bài toán 20 đoạn với 19 đoạn nhiễu, nén mềm nén mạnh hơn mọi phương pháp văn bản trong Bảng 5 nhưng chưa đạt mức suy giảm nhỏ mà bài báo gốc báo cáo trên các bộ QA với ít đoạn hơn [29]. Lọc trước bằng cross-encoder (rerank top-5, chỉ đo ở vị trí 10) giúp PISCO thêm 3,4 điểm (56,2%, khoảng tin cậy $[-0,6;\ 7,4]$, chưa có ý nghĩa) với chỉ 40 vector nhớ (72×), nhưng vẫn kém văn bản rerank top-5 trên cùng reader 6,8 điểm ($[-10,8;\ -2,8]$). Với văn bản, rerank top-5 không mất gì so với 20 đoạn (−0,2 điểm), khớp với kết quả trên Qwen2.5-7B. Lưu ý thêm: Mistral-7B nf4 đọc văn bản kém Qwen2.5-7B (60,3 so với 65,3), nên chênh lệch được tính trên cùng reader chứ không so sánh chéo.

Mô hình encode-once của nhóm. Huấn luyện 1 epoch (1.875 bước, khoảng 4 giờ GPU trên RTX 4070 SUPER, 5,3 GB VRAM) hội tụ ổn định: loss trên token đáp án giảm từ 7,4 xuống khoảng 1,2 (Hình 6). Tuy vậy, mô hình **không vượt cổng dừng** đặt trước (Mục 3.3): trên 200 câu dev, nén mềm đạt 22,0% so với 19,0% closed-book và 42,0% văn bản của Qwen2.5-1.5B gốc, trong khi cổng yêu cầu ≥29,0% và ≥29,4%. Theo kế hoạch, chúng tôi dừng P1 ở đây (không tinh chỉnh tiếp trên tập kiểm tra) và chỉ đo để phân tích.

Bảng 8 khoanh vùng nguyên nhân. (i) Nút thắt nằm ở *từng* đoạn: chỉ đưa một đoạn vàng dưới dạng 8 vector nhớ cho 23,0%, gần như bằng 20 đoạn (22,0%) và kém xa chính đoạn đó dạng văn bản (72,0%). Như vậy vấn đề không phải 19 đoạn nhiễu làm loãng, mà 8 vector không giữ được đáp án của một đoạn 128 token. (ii) Decoder đọc văn bản tốt và không bị vector nhớ của các đoạn nhiễu làm hỏng: mở đoạn vàng thành văn bản cho 70,0%, gần bằng chỉ đọc đoạn vàng (72,0%). (iii) Vector nhớ có mang *một ít* thông tin: chúng nâng decoder đã huấn luyện từ 12,0% (closed-book) lên 22,0%, và loss trên dev giảm từ 2,81 xuống 2,39 nat. Loss huấn luyện thấp (khoảng 1,2) chủ yếu đến từ các mẫu lai có đoạn vàng mở (loss 0,53, so với 1,74 ở mẫu chỉ có vector nhớ, đo lại trên mô hình đã lưu). Nói cách khác, với 15k mẫu

Bảng 7: Nén mềm so với văn bản trên *cùng* một reader (accuracy %, 500 câu/vị trí; khối Mistral: rerank chỉ đo ở vị trí 10). Token: số vị trí đầu vào của ngữ cảnh (vector nhớ, cộng token tiêu đề với mô hình của nhóm); Nén: so với 20 đoạn văn bản.

Cấu hình	@1	@10	@20	TB	Token	Nén
Mistral-7B-Instruct-v0.2 (nf4)						
Văn bản (20 đoạn)	56.6	63.2	61.2	60.3	2875	1.0×
Văn bản (rerank top-5)	–	63.0	–	–	701	4.1×
Oracle	89.6	89.6	89.6	89.6	113	25.5×
Closed-book	36.2	36.2	36.2	36.2	0	–
PISCO 16× (20 đoạn)	47.8	52.8	44.0	48.2	160	18.0×
PISCO 16× (rerank top-5)	–	56.2	–	–	40	71.9×
Qwen2.5-1.5B-Instruct						
Full (20 docs)	53.2	42.0	40.8	45.3	2875	1.0×
Rerank top-5	55.0	44.6	47.0	48.9	701	4.1×
Provence τ=0.1	58.8	51.6	48.0	52.8	621	4.6×
Rerank top-1	49.2	49.2	49.2	49.2	126	22.8×
Oracle	82.0	82.0	82.0	82.0	113	25.5×
Closed-book	26.2	26.2	26.2	26.2	0	–
Nén mềm (lai top-1)	49.0	46.8	49.8	48.5	522	5.5×
Nén mềm (20 đoạn)	26.0	26.2	25.0	25.7	416	6.9×

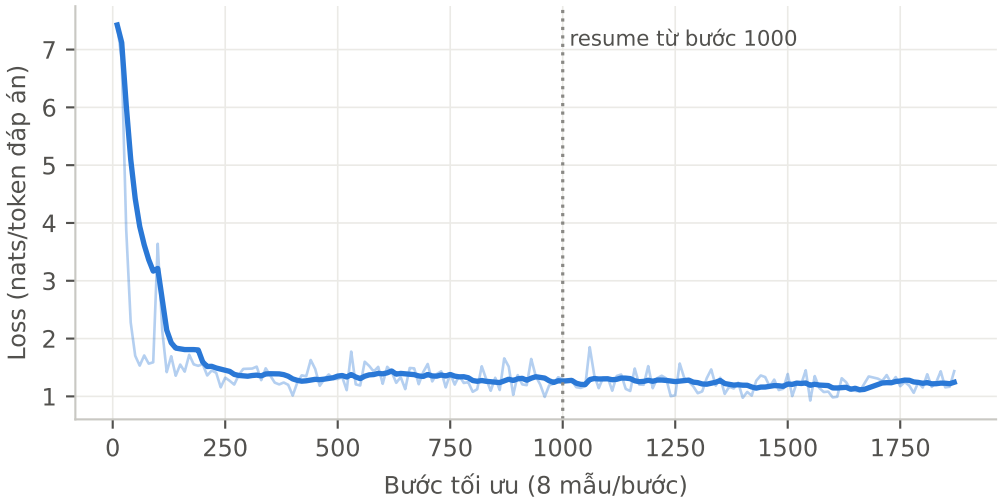
Bảng 8: Chẩn đoán cổng dừng trên 200 câu dev Tevatron (20 đoạn, accuracy %). "Đã huấn luyện": decoder có adapter; "gốc": Qwen2.5-1.5B-Instruct không adapter.

Ngữ cảnh	Decoder	Acc
Không có (closed-book)	gốc	19,0
Không có (closed-book)	đã huấn luyện	12,0
20 đoạn, tất cả là vector nhớ	đã huấn luyện	22,0
Chỉ đoạn vàng, dạng vector nhớ	đã huấn luyện	23,0
Chỉ đoạn vàng, dạng văn bản	gốc	72,0
20 đoạn, đoạn vàng mở dạng văn bản, 19 đoạn nhớ	đã huấn luyện	70,0
20 đoạn văn bản	gốc	42,0

và 1 epoch, mô hình học được cách *đọc* văn bản trong prompt trộn trước khi học được cách *nén*. Ngoài ra, closed-book của decoder đã huấn luyện tụt từ 19,0 xuống 12,0%: adapter học dựa vào ngữ cảnh thay vì kiến thức tham số.

Trên LITM (Bảng 7, khối Qwen2.5-1.5B), kết quả nhất quán với chẩn đoán. Biến thể chỉ dùng vector nhớ đạt 25,7% trung bình, ngang closed-book của reader gốc (26,2%; chênh $-0{,}2$ / 0,0 / $-1{,}2$ điểm, không có ý nghĩa). Biến thể lai đạt 48,5% với 5,5×, cao hơn 20 đoạn văn bản ở vị trí 20 (+9,0 điểm, $[4,\,2;\ 13,\,8]$) nhưng thấp hơn ở vị trí 1 ($-4{,}2$, $[-9,\,0;\ 0,\,6]$). Đối chứng quyết định là *chỉ* đưa đoạn top-1 của cross-encoder dạng văn bản: 49,2% ở mọi vị trí, không khác biến thể lai ($-0{,}2$ / $-2{,}4$ / +0,6 điểm, đều không có ý nghĩa). Toàn bộ lợi ích của biến thể lai đến từ đoạn văn bản được chọn, không phải từ 19 đoạn vector nhớ.

Nén văn bản trên reader nhỏ. Khối Qwen2.5-1.5B còn cho một kết quả phụ quan trọng. Khác reader 7B, reader 1.5B nhạy với vị trí rõ rệt (53,2 / 42,0 / 40,8, chênh 12,4 điểm; đoạn vàng càng xa đầu càng khó), và ở đây nén văn bản *làm tăng* accuracy: Provence $\tau = 0{,}1$ đạt 52,8% trung bình so với 45,3% của 20 đoạn (+5,6 / +9,6 / +7,2 điểm, cả ba có ý nghĩa), rerank top-5 tăng có ý nghĩa ở vị trí 20 (+6,2). Đây chính là hiệu ứng mà LongLLMLingua báo cáo với GPT-3.5: lợi ích "nén để chống nhiễu" không biến mất, mà phụ thuộc vào việc reader có bị nhiễu làm hỏng hay không. Lưu ý: các dòng văn bản của reader 1.5B chạy qua vLLM với prompt chuẩn (tối đa 100 token sinh), còn mô hình nén mềm sinh bằng



Hình 6: Loss huấn luyện của bộ nén mềm (trung bình 10 bước; đường đậm: trung bình trượt). Lần chạy được dừng để nhường GPU rồi tiếp tục từ trọng số ở bước 1000.

Bảng 9: Truy hồi trên kho 50,7k đoạn ($n = 500$). Hit@k: top-k chứa đoạn có đáp án (%); MRR@10 của đoạn vàng (%); kích thước chỉ mục (MB); thời gian truy vấn mỗi câu khi chạy theo batch và khi chạy từng câu (ms, GPU cho mô hình nhúng).

Hệ thống	Hit@1	Hit@5	Hit@20	MRR@10	MB	ms (batch)	ms (1 câu)
bm25	24.6	55.2	88.8	31.6	22.2	0.45	0.5
static-1024	23.4	56.6	90.8	29.3	198.1	0.49	7.8
static-256	22.6	55.8	89.6	28.6	49.5	0.25	3.8
bge-small	49.8	76.2	95.4	53.8	74.3	0.48	12.6
bge-small-hnsw	48.8	74.8	94.6	52.7	74.3	0.73	5.5
bge-small-int8	49.6	76.2	95.4	53.7	18.6	0.76	11.3
bge-small-binary	43.0	73.4	94.4	49.1	2.3	0.83	10.4

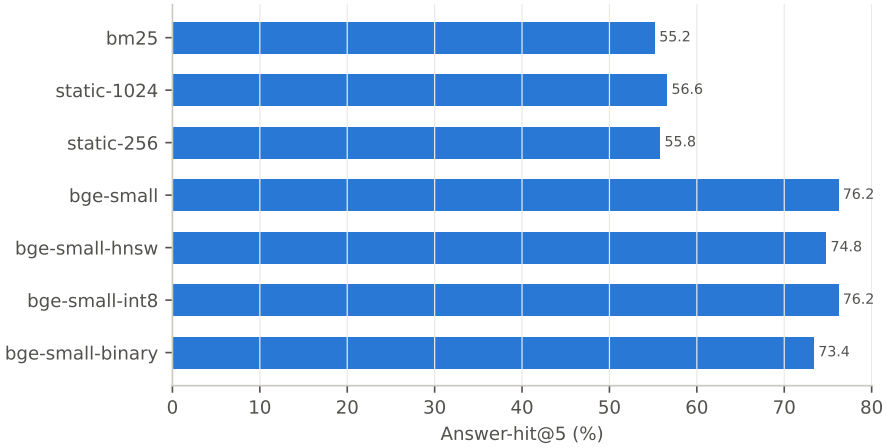
transformers với cùng prompt nhưng tối đa 48 token; với câu trả lời NQ ngắn, khác biệt này không đáng kể.

5.6 Truy hồi hiệu quả

Bảng 9 so sánh các retriever trên kho gộp 50,7k đoạn (mọi đoạn của 2.655 câu hỏi), với 500 câu hỏi của tập đánh giá.

Nhận xét.

- **Retriever dày đặc nhỏ vượt xa BM25**: bge-small (33M) đạt hit@5 76,2% so với 55,2% của BM25S, MRR@10 53,8 so với 31,6.
- **Lượng tử int8 miễn phí**: chỉ mục giảm 4× (74,3 → 18,6 MB) mà hit@5 giữ nguyên 76,2%; MRR@10 chỉ giảm 0,1 điểm.
- **Nhị phân + rescore**: chỉ mục 2,3 MB (32× nhỏ hơn) vẫn đạt 73,4% hit@5 (mất 2,8 điểm), hit@20 gần như không đổi (94,4 so với 95,4) – phù hợp làm tầng truy hồi đầu, sau đó rerank.
- **Static embedding** (không có Transformer) ngang BM25 về chất lượng (55,8–56,6%) và không cần GPU; cắt Matryoshka 1024 → 256 chiều giảm chỉ mục 4× chỉ mất 0,8 điểm hit@5.



Hình 7: Answer-hit@5 của các retriever trên kho 50,7k đoạn.

Bảng 10: RAG end-to-end: bge-small int8 top-20 trên kho 50,7k đoạn → bộ nén → Qwen2.5-7B (500 câu). Recall: ngữ cảnh sau nén còn chứa đáp án (%); Δ: chênh accuracy so với top-20 không nén, kèm khoảng tin cậy bootstrap 95%.

Cấu hình	Recall	Acc	Token	Nén	Δ [KTC 95%]
Full (20 docs)	95.4	66.0	2874	1.0×	–
LLMLingua-2 r=0.25	68.6	36.4	826	3.5×	−29.6 [−34.2; −25.0]
Rerank top-5	82.2	63.0	698	4.1×	−3.0 [−6.8; 0.8]
Provence τ=0.1	88.8	62.4	627	4.6×	−3.6 [−7.4; 0.4]

- HNSW giảm độ trễ truy vấn đơn (12,6 → 5,5 ms) đổi lấy 1,4 điểm hit@5; ở quy mô 50k đoạn, tìm kiếm chính xác vẫn đủ nhanh.

5.7 RAG end-to-end

Các mục trên dùng 20 tài liệu có sẵn của LITM, trong đó đoạn vàng luôn có mặt. Để kiểm tra kết luận trong một pipeline thật, chúng tôi ghép retriever tốt nhất về chi phí (bge-small, chỉ mục int8 18,6 MB) lấy top-20 đoạn trên kho 50,7k đoạn theo thứ tự xếp hạng, rồi nén và đọc bằng reader Qwen2.5-7B (cùng 500 câu, cùng prompt). Bảng 10 cho kết quả.

Top-20 truy hồi chứa đáp án ở 95,4% câu hỏi và cho 66,0% accuracy, ngang với 20 tài liệu LITM (65,3%): retriever 33M tham số với chỉ mục int8 không làm pipeline kém đi. Thứ tự giữa các bộ nén giữ nguyên như Bảng 5: rerank top-5 và Provence giảm token 4,1–4,6×, chỉ mất 3,0–3,6 điểm và không có ý nghĩa thống kê, còn LLMLingua-2 mất 29,6 điểm. Khác biệt duy nhất là Provence không còn hơn rerank (62,4 so với 63,0) dù giữ nhiều bằng chứng hơn (recall 88,8 so với 82,2). Một giải thích khả dĩ là kho gộp chứa nhiều đoạn gần trùng nhau về chủ đề; Provence giữ lại câu liên quan từ nhiều đoạn, nên ngữ cảnh sau nén vụn và nhiễu hơn năm đoạn nguyên vẹn của rerank. Chúng tôi chưa kiểm chứng giả thuyết này.

5.8 Chạy không cần GPU

Câu hỏi thực tế của nhóm là: nếu không có GPU, phần nào của pipeline còn dùng được? Reader được gọi qua API tương thích OpenAI (vLLM cục bộ hoặc dịch vụ trả phí), nên phần chạy cục bộ chỉ còn bộ nén. Bảng 11 cho thấy ba nhóm rõ rệt:

- **Mức tài liệu** (BM25, dense, rerank) chạy dưới 0,4 s/câu trên CPU – đủ cho ứng dụng tương tác. Rerank top-5 (196 ms) giữ 62,7% accuracy trung bình so với 65,3% của ngữ cảnh đầy đủ ở 4,1×.

Bảng 11: Độ trễ nén (trung vị ms/câu, 20 câu ở vị trí 10, gọi từng câu một sau 1 lần khởi động; không tính LLM). GPU: RTX 4070 SUPER 12 GB; CPU: Core i5-14600K, 8 luồng.

Bộ nén	GPU (ms)	CPU (ms)	CPU/GPU
BM25 top-5	2	2	0.9×
Rerank top-5	16	196	12.4×
Dense top-5	26	361	13.7×
OpenProvence τ=0.1	77	1598	20.8×
Provence τ=0.1	200	4906	24.6×
LLMLingua-2 r=0.5	455	13104	28.8×
LongLLMLingua 730t	625	14837	23.8×

Bảng 12: Phân loại thay đổi so với ngữ cảnh đầy đủ (% số cặp câu hỏi–vị trí; 1.500 cặp, BM25/Dense 500 cặp). "Sai, còn BC": tỉ lệ câu sai sau nén dù đáp án vẫn còn trong ngữ cảnh.

Bộ nén	Hỏng, mất BC	Hỏng, còn BC	Được sửa	Ròng	Sai, còn BC
Provence τ=0.1	1.7	8.4	8.8	-1.3	28.2
Rerank top-10	2.1	5.8	6.4	-1.5	26.9
Provence τ=0.01	0.6	9.0	7.2	-2.4	33.3
Rerank top-5	5.5	5.7	8.6	-2.5	20.3
OpenProvence τ=0.1	0.3	9.7	6.9	-3.1	34.5
Dense top-5	11.4	4.0	9.2	-6.2	16.6
LLMLingua-2 r=0.5	3.9	14.3	6.5	-11.7	36.9
Rerank-10 → LLMLingua-2	6.2	11.7	6.0	-11.9	29.1
BM25 top-5	19.0	3.6	5.8	-16.8	14.4
LLMLingua-2 r=0.25	13.1	19.0	5.2	-26.9	35.1
Truncate 730	27.7	4.4	4.6	-27.5	13.5

- **Mức câu** (OpenProvence, Provence) chậm 21–25× khi chuyển sang CPU: 1,6 s và 4,9 s/câu. Provence là bộ nén tốt nhất về accuracy nhưng chỉ phù hợp xử lý theo lô hoặc khi độ trễ vài giây chấp nhận được; OpenProvence (ModernBERT-base, nhỏ hơn) nhanh gấp 3 lần trên CPU.
- **Mức token** (LLMLingua-2, LongLLMLingua) chậm nhất: 13–15 s/câu trên CPU, trong khi accuracy cũng kém hơn (Mục 5.2). Không có lý do dùng chúng khi không có GPU.

Khi reader là API tính phí theo token, nén 4,6× bằng Provence giảm 78% token ngữ cảnh đầu vào (2.875 → 621) với accuracy giảm 1,3 điểm (không có ý nghĩa thống kê). Như vậy toàn bộ hệ thống – bộ nén trên CPU, reader qua API – chạy được không cần GPU; GPU chỉ rút ngắn độ trễ của các bộ nén dùng mô hình 12–29 lần. Các bộ nén trong demo (Mục 4.4) chạy trên CPU theo đúng cấu hình này.

5.9 Phân tích lỗi

Với mỗi bộ nén và mỗi vị trí, chúng tôi ghép từng câu hỏi với kết quả của ngữ cảnh đầy đủ (`scripts/error_analysis.py`) và chia thành: *hỏng do mất bằng chứng* (đầy đủ trả lời đúng, sau nén sai và đáp án không còn trong ngữ cảnh), *hỏng dù còn bằng chứng*, và *được sửa* (đầy đủ sai, sau nén đúng).

- **Mọi bộ nén đều sửa được 5–9% câu** mà ngữ cảnh đầy đủ trả lời sai: bớt nhiễu giúp reader tập trung. Với Provence $\tau = 0{,}1$, số câu được sửa (8,8%) gần bù hết số câu hỏng (10,1%).
- **Nén mức câu gần như không xoá bằng chứng** (Provence 1,7%, OpenProvence 0,3%); phần hỏng chủ yếu là reader trả lời sai dù bằng chứng còn, thường vì câu bị cắt mất ngữ cảnh xung quanh. Ví dụ câu *"who turned out to be the mother on how i met your mother"* (đáp án Tracy McConnell): sau nén, reader trả lời tên diễn viên (Cristin Milioti) thay vì tên nhân vật (Hình 2).

- **LLMLingua-2 hỏng chủ yếu *dù còn bằng chứng***: ở $r = 0{,}5$, 14,3% so với 3,9% do mất bằng chứng; ở $r = 0{,}25$, 19,0% so với 13,1%. Văn bản bị bỏ token rời rạc (mất mạo từ, giới từ, dấu câu) vẫn chứa đáp án nhưng reader 7B không còn đọc đúng quan hệ giữa các thực thể.
- **Truncate và BM25 hỏng do mất bằng chứng** (27,7% và 19,0%).
- **Một phần lỗi do cách chấm**: reader 7B trả lời dài (tối đa 100 token) và đổi định dạng, ví dụ "September 8, 2010" bị chấm sai với đáp án "8 September 2010". Lỗi này ảnh hưởng như nhau đến mọi cấu hình nên không làm sai lệch so sánh, nhưng làm accuracy tuyệt đối thấp hơn thực tế.

6 Kết luận

6.1 Trả lời câu hỏi nghiên cứu

RQ1. Ở mức nén ≈4×, **chọn lọc theo câu** (Provence $\tau = 0{,}1$, 4,6×) là cách duy nhất không làm giảm accuracy có ý nghĩa thống kê ở cả ba vị trí (64,0% so với 65,3% của ngữ cảnh đầy đủ). Chọn lọc theo tài liệu (rerank top-5) rẻ hơn nhiều nhưng mất 2,6 điểm và giảm có ý nghĩa khi đoạn vàng ở cuối. Xoá theo token (LLMLingua-2 r=0,25) làm accuracy rơi về mức closed-book (38,3% so với 38,8%): phân tích lỗi cho thấy phần lớn thiệt hại đến từ văn bản bị xé vụn chứ không phải mất đáp án. Với reader hiện đại, đường cong vị trí vốn đã gần phẳng (chênh 5,8 điểm) nên nén không còn tác dụng "làm phẳng" như báo cáo cho GPT-3.5.

RQ2. Ở quy mô 50,7k đoạn, retriever dày đặc nhỏ (bge-small, 33M tham số) vượt BM25 21 điểm hit@5; lượng tử int8 giảm chỉ mục 4× không mất chất lượng, nhị phân + rescore giảm 32× chỉ mất 2,8 điểm. Ghép thành pipeline end-to-end (top-20 bge-small int8 → nén → reader 7B), accuracy đạt 66,0%, ngang 20 tài liệu LITM; rerank top-5 hoặc Provence giảm 4× token chỉ mất 3,0–3,6 điểm (không có ý nghĩa).

RQ3. Tái lập được hai kết luận của các công bố: BM25 kém xa dense, và LLMLingua-2 dùng đơn lẻ ở 4× sụt mạnh (−21 đến −30 điểm so với −11 đến −27 được công bố). Không tái lập được kết luận "nén làm tăng accuracy khi đoạn vàng ở giữa" của LongLLMLingua (+17,1 điểm với GPT-3.5, +1,0 không có ý nghĩa với Qwen2.5-7B): hiệu ứng đó phụ thuộc vào reader bị *lost in the middle*, điều không còn đúng với LLM mở 7B thế hệ mới ở độ dài 3K token. Với reader nhỏ 1.5B vốn nhạy vị trí, hiệu ứng này xuất hiện trở lại: Provence tăng accuracy có ý nghĩa ở cả ba vị trí (Mục 5.5).

RQ4. Có. Bộ nén chạy trên CPU (rerank 0,2 s/câu, Provence 4,9 s/câu), reader gọi qua API; nén giảm 78% token đầu vào và do đó giảm chi phí API tương ứng. GPU chỉ cần để rút ngắn độ trễ nén.

RQ5. Chưa, ở quy mô của nhóm. Checkpoint công bố PISCO nén 18× nhưng mất 9–17 điểm so với văn bản trên cùng reader Mistral-7B (48,2% so với 60,3%). Mô hình encode-once của nhóm (Qwen2.5-1.5B, 15k mẫu, 1 epoch) không vượt cổng dừng đặt trước: chỉ dùng vector nhớ thì ngang closed-book (25,7% so với 26,2% trên LITM), vì 8 vector không giữ được đáp án của từng đoạn (23% so với 72% khi đọc đoạn vàng dạng văn bản). Biến thể lai đạt 48,5% ở 5,5×, nhưng đúng bằng chỉ đọc đoạn top-1 dạng văn bản (49,2%). Ngược lại, trên reader 1.5B nhạy với vị trí, nén *văn bản* bằng Provence tăng accuracy có ý nghĩa (+7,5 điểm trung bình). Với ngân sách nhỏ, chọn lọc văn bản vẫn là lựa chọn tốt hơn nén mềm.

6.2 Hạn chế

- Chỉ một bộ dữ liệu (NQ-Open, câu trả lời ngắn) và một reader chính; kết luận về nén mức token có thể khác với tác vụ cần lập luận nhiều bước hoặc ngữ cảnh dài hơn nhiều (32K+).
- Chỉ số *best subspan EM* phạt các câu trả lời đúng nhưng khác định dạng (ví dụ ngày tháng); một phần lỗi trong Mục 5.9 thuộc loại này.

- LongLLMLingua chỉ chạy 200 câu ở một vị trí vì chi phí; các số liệu GPT-3.5 chỉ dùng để so sánh độ chênh, không so sánh tuyệt đối.
- Mô hình nén mềm của nhóm được huấn luyện 1 epoch trên 15k mẫu với một GPU 12GB, ít hơn nhiều so với dữ liệu và tài nguyên của PISCO; kết luận thất bại chỉ đúng ở quy mô này (một epoch, 8 vector/đoạn, LLM 1.5B).

6.3 Hướng phát triển

- Kết hợp hai nhánh hiệu quả nhất: chọn lọc theo câu (Provence) cho tài liệu được xếp hạng cao và bộ nhớ nén cho phần còn lại – tương tự cơ chế trộn token chọn lọc và token nén trong attention thưa của DeepSeek.
- Đánh giá trên ngữ cảnh dài hơn (50–100 tài liệu) và các bộ dữ liệu đa bước (HotpotQA, MuSiQue), nơi reader còn bị ảnh hưởng bởi vị trí.
- Chưng cất Provence sang mô hình nhỏ hơn để giảm độ trễ trên CPU xuống dưới 1 s/câu.

Phân công công việc

Thành viên	Công việc
Nguyễn Hoàng Sơn	Track A – nén ngữ cảnh: pipeline đánh giá dùng chung (dữ liệu, chỉ số, đếm token, lược đồ kết quả), tích hợp các bộ nén cứng, chạy reader qua vLLM, kiểm định ý nghĩa thống kê, phân tích lỗi; nén mềm (PISCO, mô hình encode-once của nhóm); báo cáo các phần Phương pháp, Cài đặt, Kết quả.
Võ Việt Hoàng	Track B – truy hồi hiệu quả (BM25, dense, lượng tử hoá, Matryoshka, HNSW); đo độ trễ CPU/GPU; demo Gradio; báo cáo các phần Giới thiệu, Nghiên cứu liên quan, Kết quả truy hồi; slide.
Cả hai	Thống nhất giao thức thí nghiệm và số liệu, đọc chéo báo cáo, chuẩn bị thuyết trình.

Tiến độ từng việc được ghi trong `docs/tasks.md` của kho mã.

Tài liệu

[1] Tom Aarsen. Train 400x faster static embedding models with sentence transformers. Hugging Face Blog, https://huggingface.co/blog/static-embeddings, 2025.

[2] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. xRAG: Extreme context compression for retrieval-augmented generation with one token. In *Advances in NeurIPS*, 2024. arXiv:2405.13792.

[3] Xin Cheng, Rui Tian, Wangding Zeng, Damai Dai, et al. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026.

[4] Nadezhda Chirkova, Thibault Formal, Vassilina Nikoulina, and Stéphane Clinchant. Provence: efficient and robust context pruning for retrieval-augmented generation. In *Proceedings of ICLR*, 2025. arXiv:2501.16214.

[5] Ziqiang Cui et al. Less is more: Elevating RAG via performance-driven context compression, 2025. ICML 2026, arXiv:2508.19282.

[6] DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models, 2025.

[7] DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence, 2026.

[8] DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression, 2026.

[9] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library, 2024. arXiv:2401.08281.

[10] EmbeddingGemma Team. EmbeddingGemma: Powerful and lightweight text representations, 2025. arXiv:2509.20354.

[11] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Tevatron: An efficient and flexible toolkit for dense retrieval, 2022.

[12] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations (ICLR)*, 2024.

[13] Jie He, Richard He Bai, Sinead Williamson, Jeff Z. Pan, et al. CLaRa: Bridging retrieval and generation with continuous latent reasoning, 2025.

[14] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[15] Taeho Hwang, Sukmin Cho, Soyeong Jeong, Hoyun Song, SeungYoon Han, and Jong C. Park. EXIT: Context-aware extractive compression for enhancing retrieval-augmented generation. In *Findings of ACL*, 2025. arXiv:2412.12559.

[16] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, 2023. arXiv:2310.05736.

[17] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of ACL*, 2024. arXiv:2310.06839.

[18] Jiajie Jin, Yutao Zhu, Xinyu Yang, Chenghao Zhang, and Zhicheng Dou. FlashRAG: A modular toolkit for efficient retrieval-augmented generation research. In *Companion Proceedings of the ACM Web Conference (WWW)*, 2025. arXiv:2405.13576.

[19] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.

[20] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, and Ali Farhadi. Matryoshka representation learning. In *Advances in NeurIPS*, 2022. arXiv:2205.13147.

[21] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019.

[22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of SOSP*, 2023.

[23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in NeurIPS*, 2020.

[24] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of EMNLP*, 2023. arXiv:2310.06201.

[25] Zongqian Li, Yinhong Liu, Yixuan Su, and Nigel Collier. Prompt compression for large language models: A survey. In *Proceedings of NAACL*, 2025. arXiv:2410.12388.

[26] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of MLSys*, 2024.

[27] Xiaoqiang Lin et al. REFRAG: Rethinking RAG based decoding, 2025. arXiv:2509.01092.

[28] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. arXiv:2307.03172.

[29] Maxime Louis, Hervé Déjean, and Stéphane Clinchant. PISCO: Pretty simple compression for retrieval-augmented generation, 2025. arXiv:2501.16075.

[30] Maxime Louis, Thibault Formal, Hervé Déjean, and Stéphane Clinchant. OSCAR: Online soft compression and reranking, 2025. ICLR 2026, arXiv:2504.07109.

[31] Xing Han Lù. BM25S: Orders of magnitude faster lexical search via eager sparse scoring, 2024. arXiv:2407.03618.

[32] Youssef Mohamed, Mohamed Elhoseiny, et al. XProvence: Zero-cost multilingual context pruning for retrieval-augmented generation, 2026. ECIR 2026, arXiv:2601.18886.

[33] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of ACL*, 2024. arXiv:2403.12968.

[34] Qwen Team. Qwen2.5 technical report, 2024. arXiv:2412.15115.

[35] David Rau, Shuai Wang, Hervé Déjean, and Stéphane Clinchant. Context embeddings for efficient answer generation in RAG. In *Proceedings of WSDM*, 2025. arXiv:2407.09252.

[36] Aamir Shakir, Tom Aarsen, and Sean Lee. Binary and scalar embedding quantization for significantly faster & cheaper retrieval. Hugging Face Blog, https://huggingface.co/blog/embedding-quantization, 2024.

[37] Yuichi Tateno. OpenProvence: open-source context pruning models. https://github.com/hotchpotch/open_provence, 2025. Hugging Face: hotchpotch/open-provence-reranker-v1-gte-modernbert-base.

[38] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general Chinese embeddings. In *Proceedings of SIGIR*, 2024. arXiv:2309.07597 (BGE models).

[39] Fangyuan Xu, Weijia Shi, and Eunsol Choi. RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation. In *Proceedings of ICLR*, 2024. arXiv:2310.04408.

[40] Chanwoong Yoon, Taewhoo Lee, Hyeon Hwang, Minbyul Jeong, and Jaewoo Kang. CompAct: Compressing retrieved documents actively for question answering. In *Proceedings of EMNLP*, 2024. arXiv:2407.09014.

[41] Yanzhao Zhang et al. Qwen3 Embedding: Advancing text embedding and reranking through foundation models, 2025. arXiv:2506.05176.

[42] Yong Zhang et al. Sentinel: Decoding context utilization via attention probing for efficient LLM context compression, 2025. arXiv:2505.23277.

A Tái lập kết quả

Mọi số liệu trong báo cáo được sinh từ các tệp `results/` trong kho mã; bảng và hình được tạo lại bằng một lệnh. Các bước dưới đây chạy trên một GPU 12GB; mọi bộ nén cũng chạy được trên CPU (`EFFRAG_DEVICE=cpu`).

```
uv sync && cp .env.example .env                    # LLM_BASE_URL,
    LLM_MODEL
uv run python scripts/download_data.py             # LITM NQ-Open 20
    tai lieu
# reader (vLLM, 4-bit AWQ)
vllm serve Qwen/Qwen2.5-7B-Instruct-AWQ --served-model-name qwen2
    .5-7b-instruct \
      --max-model-len 4096 --seed 0
bash scripts/run_all_compression.sh --dry-run      # chi nen: recall,
     ti le nen
bash scripts/run_all_compression.sh                # goi LLM (co
    cache theo hash prompt)
uv run python scripts/run_retrieval.py             # Track B
EFFRAG_DEVICE=cpu  uv run python scripts/bench_device.py --n 20
EFFRAG_DEVICE=cuda uv run python scripts/bench_device.py --n 20
uv run python scripts/eval_pisco.py --n 500        # PISCO (Mistral-7
    B nf4)
uv run python scripts/build_soft_train.py          # du lieu huan
    luyen nen mem
uv run python scripts/train_soft.py --out data/soft/run1
uv run python scripts/eval_soft.py gate --run data/soft/run1    #
    cong dung
uv run python scripts/eval_soft.py diag --run data/soft/run1    #
    chan doan
uv run python scripts/eval_soft.py litm --run data/soft/run1
bash scripts/run_reader_1p5b.sh                    # dong van ban cua
     reader 1.5B (vLLM)
uv run python scripts/build_retrieved.py           # Track C: top-20
    bge-small int8
uv run python scripts/run_compression.py --retriever bge-small-int8
     --method provence \
      --params '{"threshold": 0.1}'                # tuong tu: full,
    rerank_topk, llmlingua2
uv run python scripts/make_figures.py              # bang + hinh cho
    bao cao/slide
uv run python scripts/error_analysis.py
uv run python app/demo.py                          # demo Gradio
uv run python scripts/overleaf_zip.py              # zip bao cao/
    slide cho Overleaf (XeLaTeX)
```

B Prompt

Mọi bộ nén dùng chung prompt gốc của Liu et al. [28]; mỗi tài liệu được hiển thị dưới dạng `Document [i](Title: ...) <văn bản>` và đưa vào lượt `user` của chat template:

```
Write a high-quality answer for the given question using only the
   provided
search results (some of which might be irrelevant).

Document [1](Title: ...) ...
...
Document [20](Title: ...) ...

Question: {question}
Answer:
```

Closed-book dùng `Question: {question}\nAnswer:`. PISCO dùng prompt riêng của mô hình (chỉ dẫn hệ thống gộp vào lượt `user` vì Mistral không có vai trò `system`); hàng "văn bản" của PISCO dùng đúng prompt đó với tài liệu dạng văn bản để so sánh cùng điều kiện.

C Siêu tham số mô hình nén mềm

Bảng 13: Siêu tham số huấn luyện bộ nén mềm encode-once của nhóm.

Tham số	Giá trị
Mô hình nền	`Qwen/Qwen2.5-1.5B-Instruct` (bf16)
Adapter	2 LoRA (encoder, decoder), $r = 16$, $\alpha = 32$, dropout 0,05, mọi lớp tuyến tính
Vector nhớ / tài liệu	8 (tài liệu cắt ở 128 token, tức nén 16×)
Chiếu encoder → decoder	bỏ 8 kênh massive activation, tuyến tính khởi tạo đơn vị, chuẩn hoá L2, nhân hệ số học được
Dữ liệu	15.000 câu NQ (Tevatron), $k \in \{5, 10, 20\}$ đoạn, âm tính khó BM25
Chế độ lai	50% mẫu giữ 1 đoạn dạng văn bản (đoạn vàng với xác suất 0,6)
Tối ưu	AdamW, lr 10^{-4}, warmup 50 bước, cosine, clip 1,0
Batch	8 mẫu/bước (tích luỹ gradient), 1 epoch ≈ 1.875 bước
Bộ nhớ	gradient checkpointing; ≈5GB VRAM